

Spark Performance Analysis: Tweet Sentiment Pipeline

DSCC-402 Final Project

Natnael Ayalew

Overview

My pipeline ingests roughly 42K tweets from S3, explodes @mentions to produce 44K rows, runs each cleaned tweet through a RoBERTa sentiment classifier loaded from Unity Catalog, and aggregates results per mention. The ML pipeline is built with Spark Declarative Pipelines on Databricks serverless compute, with a downstream materialized view feeding a dashboard. Looking at the performance tab from the relevant end-to-end runs and model evaluation metrics:

Tables 4

Performance 4

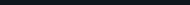
Q Filter by table

Type ▾

Status ▾

Name ↕	Catalog	Schema	Type	Duration	Output re...	Ex
 gold_tweet_aggregations	 workspace	default	Materialized view	2s	0	-
 tweets_bronze	 workspace	default	Streaming table	20s	42K	-
 tweets_gold	 workspace	default	Streaming table	2s	0	-
 tweets_silver	 workspace	default	Streaming table	11s	44K	-

 May 01, 2026, 09:24 PM



 1m 14s

 Refresh all

 [Query performance](#)

Tables 4

Performance 4

Filter by table

Type

Status

Name	Catalog	Schema	Type	Duration	Output re...
gold_tweet_aggregations	workspace	default	Materialized view	3s	18K
tweets_bronze	workspace	default	Streaming table	12s	-
tweets_gold	workspace	default	Streaming table	2h 1m 8s	44K
tweets_silver	workspace	default	Streaming table	1s	-

May 02, 2026, 10:18 AM

2h 2m 38s

Refresh all

[Query performance](#)

Metrics (8)	
Search metrics	
Metric	Latest
accuracy	0.735529239297249
f1_macro	0.7262373825780161
f1_negative	0.6758016510361095
f1_positive	0.7766731141199227
precision_negative	0.8653196232113324

Total runtime was 2 hours, with gold accounting for over 99% of wall-clock time. That single observation drives most of what follows. The pipeline works correctly, but it's slow because the

model inference step is basically doing all the work. Here are three optimization areas to look at: shuffle, skew, and serialization.

Shuffle

The aggregation stage in the application layer does a GROUP BY mention over the gold table, which forces a shuffle. The performance tab shows this stage reading 20,950 rows, with 839 KB shuffled in and 497 KB shuffled out, finishing in 3.3 seconds. So at current scale this isn't a bottleneck — it's tiny. That said, the default partition of 200 is clearly excessive here. With ~21K rows distributed across 200 partitions, average partition size is about 100 rows. That's well below the threshold where parallelism handles best scheduling overhead — Spark spends more time scheduling tasks than processing them. In addition to tuning shuffle partitions, Adaptive Query Execution (AQE) could auto-coalescing these post-shuffle.

Skew

The mention distribution in this dataset is heavily skewed. The dashboard's Top 20 Positive Mentions chart shows @mileycyrus with a sentiment_power of 65, while the median mention has total counts in the single digits — a significant difference between the most-mentioned user and a typical one. This is the kind of distribution where one would expect skew problems in a groupBy or join operation. The data volume here is probably small enough that skew exists structurally but doesn't manifest as a runtime issue. If the incoming tweet were in the 100s of millions for the same pipeline, appending a small random suffix to skewed keys before aggregation, then re-aggregating (salting) would ideally help even though it adds query complexity.

Serialization

This is the optimization that would directly impact the runtime. Gold processed 43,658 rows incoming from silver in 2 hours, which works out to roughly 166 milliseconds per row. The extra time is likely Python - JVM serialization overhead. Each batch of rows in the gold layer has to be arrow-serialized from Spark's internal format into Python, run through the sentiment Python UDF model, then serialized back out as a struct. mlflow.pyfunc.spark_udf already uses pandas batching internally, which is the right pattern, but the per-batch overhead dominates on the serverless compute assigned given batches end up small enough. What I'd consider for the gold layer is switching from the UDF to a Spark-native serving framework that runs entirely on the JVM and avoids the Python boundary altogether. Getting a dedicated GPU compute also helps with reducing inference runtime.